

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Arunprasath R 192524077(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Arunprasath R

Q. No	Problem Understanding(20)	Method Selection(20)	Solution Process(25)	Accuracy of Calculation (20)	Interpretation of Results(15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Q1. Smart Electricity Billing System

Concepts Covered: Intermediate Languages, Assignment Statements, Optimization

Problem Statement

An electricity billing system calculates the total payable amount using units consumed, rate per unit, and tax.

$$\text{energy_cost} = \text{units} \times \text{rate}$$
$$\text{tax_amount} = \text{energy_cost} \times \text{tax_rate}$$
$$\text{total_bill} = \text{energy_cost} + \text{tax_amount}$$

Tasks

- (i) Write suitable declarations for all variables with appropriate data types.
- (ii) Generate three-address intermediate code for the billing calculation.
- (iii) Represent the intermediate code using quadruples, triples, and indirect triples.
- (iv) Identify common subexpressions and optimization opportunities.
- (v) Produce the optimized intermediate code with justification. (10 Marks)

CO4 AT2

Aswinprasad R

192524097

CSA1403

Computer Design

1) Smart electricity billing system:-

Given :-

energy = unit * rate

tax = energy * tax-rate

total = energy + tax.

i) declaration:-

int unit;

float rate, tax-rate;

float energy, tax, total;

ii) 3-address code:-

$t_1 = \text{units} * \text{rate}$

energy = t_1

$t_2 = t_1 * \text{tax-rate}$

tax = t_2

$t_3 = t_1 + t_2$

total = t_3

iii) quadruples.

0	*	units	rate	t_1
1	*	t_1	tax-r.	t_2
2	+	t_1	t_2	t_3
3	=	t_1	-	energy
4	=	t_2	-	tax
5	=	t_3	-	total

iv) Triples:

0	*	units	rate
1	*	(0)	tax-rate
2	+	(1)	(1)

v) optimization:- Exp: units * rate.

code:- $t_1 = \text{units} * \text{rate}$
 $t_2 = t_1 * \text{tax-rate}$
 $t_3 = t_1 + t_2$

Q2. Employee Promotion Evaluation System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

An employee is eligible for promotion if:

Experience $\geq$ 5 years

AND (Performance Score $>$ 90 OR Certification = TRUE)

Tasks

- (i) Formulate the Boolean expression.
- (ii) Generate intermediate code using short-circuit evaluation.
- (iii) Apply backpatching to resolve forward jumps.
- (iv) Generate the final labelled intermediate code.
- (v) Draw the control-flow graph for the evaluation process. (10 Marks)

2) Employee Promotion Evaluation System:-

condition:

$exp \geq 5$ AND
(performance > 90 OR certification == TRUE).

(i) Boolean exp:

eligible =

$(exp \geq 5)$ AND

$(performance > 90)$ OR $(certification == TRUE)$.

(ii) short circuit TAC.

L1: if $exp > 5$ goto L2
goto Lfalse

L2: if performance > 90 goto Ltrue
goto L3.

L3: if certification == TRUE goto Ltrue.
goto false.

Ltrue: eligible = TRUE
goto Lend.

Lfalse: eligible = FALSE
Lend:

(iii) Backpatching:

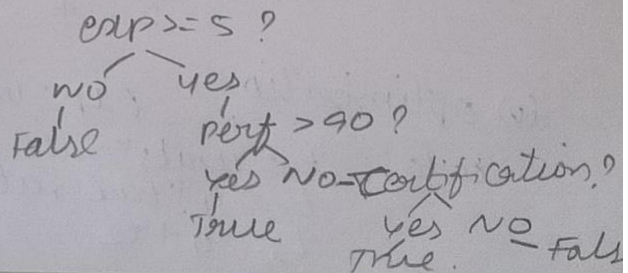
Initial

if $exp \geq 5$ goto L2
goto Lfalse

if performance > 90 goto Ltrue
goto L3

if certification == TRUE goto Ltrue
goto Lfalse

iv) control flow:



Q3. Hospital Patient Priority System

Concepts Covered: Case Statements, Intermediate Languages

Problem Statement

A hospital assigns treatment priority based on emergency level.

```
switch(priority)

{

case 1: CRITICAL;

case 2: HIGH;

case 3: MEDIUM;

case 4: LOW;

}
```

Tasks

- (i) Convert the switch statement into three-address code using conditional jumps.
 - (ii) Generate an equivalent jump-table implementation.
 - (iii) Represent the implementation using quadruples.
 - (iv) Compare conditional branching with jump-table execution.
 - (v) Recommend the most suitable implementation for emergency hospitals with justification.
- (10 Marks)

3) Hospital Patient Priority System.

(i) Conditional jump JAL:

```
if priority = 1 goto L1
    "      == 2 goto L2
    "      == 3 goto L3
    "      == 4 goto L4.
goto Ldefault.
```

L1: CRITICAL
goto Lend

L2: HIGH
goto Lend

L3: LOW
goto Lend

Ldefault:
INVALID PRIORITY
Lend:

(ii) Jump table jmp:

```
t1 = priority - 1
if t1 < 0 goto Ldefault
if t1 > 3 goto default
goto JumpTable [t1].
```

(iii) Quadruples:

no	op	Arg1	Arg2	Result
0	if ==	pri	1	L1
1	if ==	pri	2	L2
2	if ==	pri	3	L3
3	if ==	pri	4	L4
4	goto	-	-	Ldefault.

(iv) Comparison:

conditional branching

Jump table.

- Performs comparison - directly select target
- Simple to implement - faster for dense cases
- Suitable for few cases - many consecutive cases

Q4. E-Commerce Payment Processing System

Concepts Covered: Procedure Calls

Problem Statement

An online shopping application processes payments through the following procedures.

amount = calculateAmount(cart)

processPayment(amount)

sendConfirmation(amount)

Tasks

- (i) Generate intermediate code.
- (ii) Illustrate parameter passing.
- (iii) Design the activation record.
- (iv) Draw stack diagrams for call and return operations.
- (v) Explain how procedure calls are managed during execution. (10 Marks)

4) E-commerce payment processing system:

(i) Intermediate code:-

```
param cart
t1 = call calculate amount 1
amount = t1
paramount
call process payment, 1
param amount
call send confirmation 1.
```

(ii) parameter passing:-

calculate Amount:-

```
param cart
t1 = call calculate amount, 1
```

process payment:-

```
param amount
call process payment, 1
```

(iii) Activation record:-

- a) procedure AR contains parameters, return address, old frame pointer, saved registers, local variables, temporaries.

iv) Procedural-call:-

- a) Parameter are passed.
- b) Return address is saved
- c) AR is created.
- d) control transfer to called procedure
- e) Procedure executes
- f) Return value is produced
- g) AR is removed
- h) control returns to caller.

Q5. Smart ATM Withdrawal Verification System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A customer can withdraw cash if:

Account Balance $\geq$ Withdrawal Amount

AND (PIN Verified = TRUE OR Biometric Verified = TRUE)

Tasks

- (i) Write the Boolean expression.
- (ii) Generate short-circuit intermediate code.
- (iii) Apply backpatching.
- (iv) Generate the final labelled intermediate code.
- (v) Draw the control-flow graph. (10 Marks)

5) smart ATM withdrawal Verification system:-

(i) Boolean exp:-

withdraw =

(Bal >= Amount) AND
(PIN == TRUE OR Biometric == TRUE)

ii) Short circuit TAC:-

L1:

if Bal >= Amount goto L2
goto Lfalse

L2:

if PIN == TRUE goto Ltrue
goto L3.

L3:

if Biometric == TRUE goto Ltrue
goto Lfalse

Ltrue:

Allow withdrawal
goto Lend

Lfalse:

deny
Lend;

(iii) Backpatch:

if Bal >= Amount goto L2
goto Lfalse

L2: if PIN == TRUE goto Ltrue
goto L3

L3: if Biometric == TRUE goto Ltrue
goto Lfalse.

iv) control flow:-

Balance >= Amount ?

NO

deny

yes

PIN = TRUE ?

yes

Allow

NO

Biometric ?

yes

Allow

NO

deny

v) conclusion:-

SC evaluation avoids
unnecessary checks.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation